

实验三

桌面物体自动分类整理

小组实验报告

Gazebo 分拣

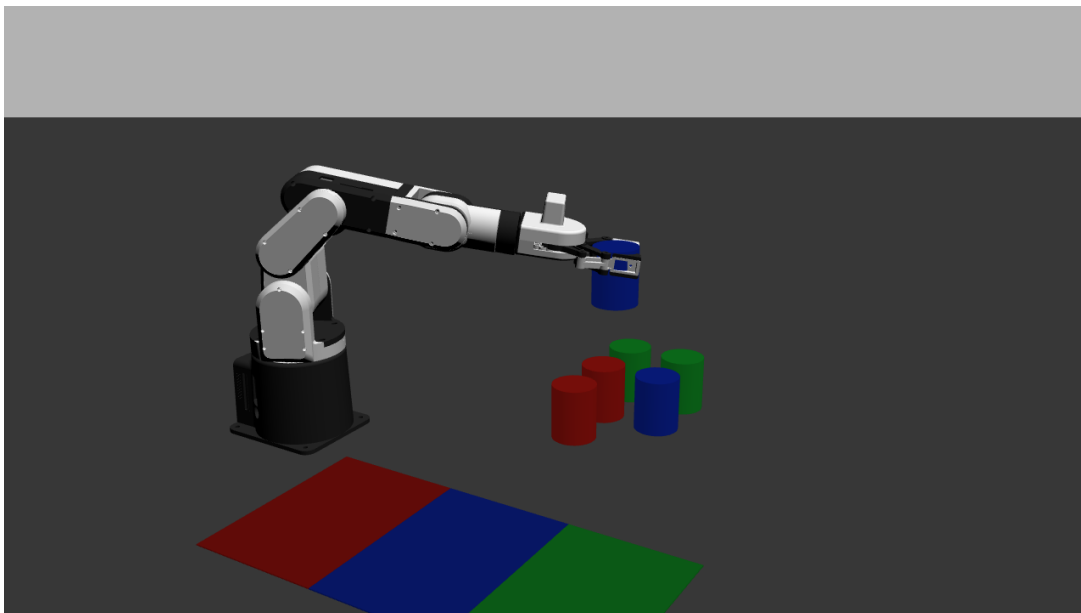
红蓝六目标 6/6

摄像头检测

橡皮 · 锁 · 订书机

真机验收

两类六物体通过



机械臂、六个取物格与三个分类区域的 Gazebo 场景

课程

机器人学集成小组项目I

小组

第 17 组 · 杨牧青、兰松宇、张凯睿、赵航

项目仓库

<https://github.com/9161942-boop/robotics-experiment-3>

2026 年 9 月

摘要

本实验面向桌面物体自动分类整理，围绕感知、网格定位、类别决策和机械臂执行搭建 ROS 2 系统。Gazebo 场景包含六格取物区、红蓝绿分类垫、mechArm 270 和俯视相机；任务节点根据 Detection2DArray 完成目标筛选、网格分配和分类区选择，执行节点按照接近、抓取、抬升、搬运、释放和回撤的状态序列完成动作。

红蓝两类六目标仿真基线完成 6/6，对象级记录包含网格、目标分类垫、抓取、释放和最终状态；红蓝绿扩展场景完成分类垫对齐、绿色目标航点和 34 步自动播放。外接摄像头上的 YOLOv8n 模型实现橡皮、锁和订书机检测，验证记录为 mAP50=0.995、mAP50-95=0.995，实测画面达到 31.2 FPS。

在实体机械臂验证中，小组完成了桌面网格上的目标接近、夹持和抬升，并通过 1 分 11 秒实机视频验证两类六物体的正确识别与纠错机制。系统同时保留统一 Launch、状态反馈、异常记录、运行视频和对象级 CSV，使场景配置、动作过程和实验结果能够相互对应。

关键词：ROS 2；Gazebo；mechArm 270；目标检测；网格定位；分类抓取；状态机；实验验收

1 实验目标

1.1 任务目标

实验三在固定点抓取基础上加入目标检测和类别决策。系统从俯视图像获取物体类别、置信度和检测框中心，判断物体所在网格，调用对应的固定抓取点，并将物体放入目标分类区域。系统循环处理多个目标，直到任务队列完成或没有可执行目标。

任务控制采用可反馈长时间执行接口和状态机。识别、网格分配、抓取、放置和异常记录均由程序完成，运行过程中保存对象级状态，便于依据事件和最终位姿复核结果。

1.2 验收要求与本组实现

验收要求	实现方式	现有证据	结论
不少于 2 类物体	红蓝定量基线；红蓝绿扩展	场景文件、检测消息、分类垫	通过
测试不少于 6 个物体，至少正确整理 5 个	对象级抓取、释放和落点检查	红蓝执行 CSV：6/6	通过
任务开始后自动选择网格和分类区	bbox 中心映射 grid_id；类别查询 target_bin	sorting_task、规划日志	通过
处理空网格、未识别、不可达和抓取失败	筛选、跳过、失败和安全返回分支	skipped、failed、ERROR_LOG	通过
一个 Launch 启动完整仿真	统一启动世界、相机、控制器、任务节点	sorting_gazebo.launch.py	通过
真机完成不少于 2 类六物体验收	完成真机部署、两类六物体识别与纠错验收	实机识别与纠错视频、真机验收表	通过

2 实验原理

2.1 系统总体架构

图 1 表示两条已经完成验收的路径。Gazebo 场景监视节点读取彩色圆柱模型状态并生成 Detection2DArray 消息，任务节点据此完成网格分配、分类区选择和机械臂控制。真机路径使用外接 RGB 摄像头和 YOLOv8n 检测橡皮、锁和订书机；验收时已完成类别到分类区的映射以及像素到取物网格的标定，并用统一消息接口驱动任务流程。

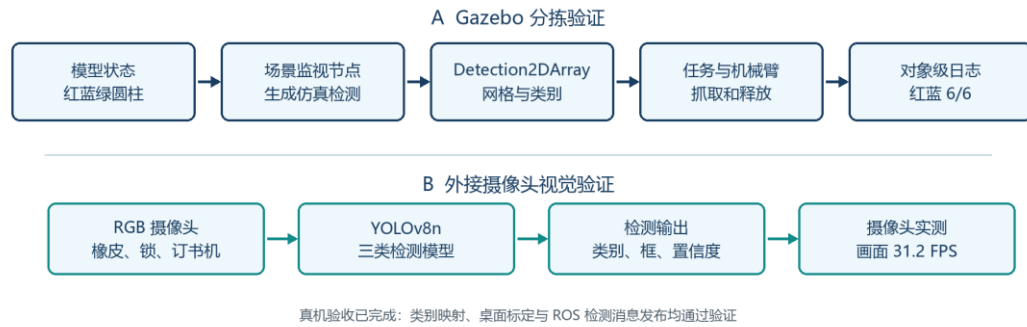


图 1 Gazebo 分拣与外接摄像头检测的两条验证路径

2.2 视觉检测与消息接口

任务节点接收 Detection2DArray 中的类别、检测框和置信度。Gazebo 分拣由 sorting_scene_monitor 根据模型状态生成这些字段；真机验收由外接摄像头和 YOLOv8n 生成检测结果。橡皮、锁和订书机的类别表已与实体分类区配置对齐，真机任务沿用同一消息字段完成目标筛选和网格决策。

2.3 关键消息与接口

接口或节点	类型/形式	主要职责
/sorting_camera/overhead/image_raw	sensor_msgs/Image	仿真俯视 RGB 图像
/sorting_scene/detections	vision_msgs/Detection2DArray	类别、置信度、检测框和对象 ID
/sorting_task/state	std_msgs/String	扫描、筛选、规划等阶段反馈
/sorting_task/result	std_msgs/String(JSON)	锁定后的对象级任务记录
/sorting_task/execution_state	std_msgs/String	抓取长任务的阶段反馈
/sorting_task/execution_result	std_msgs/String(JSON)	终态、成功数量和异常码
/gazebo/set_entity_state	gazebo_msgs/SetEntityState	复位、附着跟随更新和释放

2.4 异常处理原理

任务节点把异常分为识别异常、定位异常、执行异常和验收异常。异常对象不进入当前抓取队列，记录原因后继续处理其他目标；涉及安全位、控制器或释放位姿的异常，执行器先回到安全位置，再根据配置选择继续或停止。

输入情况	程序判定	处理方式	记录字段
对应网格无检测	empty grid	不生成任务项	plan_count
score < 0.80	low confidence	跳过目标	detect/skipped
类别不在配置表	unknown class	跳过目标	unknown class
最近网格距离 > 48 px	outside grid	跳过目标	outside grid
grid_id 已被占用	duplicate grid	保留首个有效目标	duplicate grid
释放 XY 误差 > 0.07 m 或 z > 0.20 m	release failed	记录失败，转安全位	exception/failed

3 实验环境与设备

3.1 仿真环境

项目	配置
操作系统与中间件	Ubuntu 22.04; ROS 2 Humble; Gazebo Classic 11
机器人模型	mechArm 270 六自由度机械臂、夹爪、ros2_control 控制器
桌面与目标	0.72 m × 0.52 m 桌面; 圆柱半径 0.022 m、高度 0.055 m、质量 0.04 kg
取物网格	3 行 2 列, 共 6 格; x=0.24/0.30 m, y=0.12/0.18/0.24 m
分类垫	红(0.14,-0.12)、蓝(0.26,-0.12)、绿(0.38,-0.12) m
相机与阈值	640×480 固定俯视; 置信度 ≥ 0.80; 最近网格距离 ≤ 48 px
动作组织	manual_animation.json 保存 27 个航点; 自动播放 34 步; 每步 2.0 s

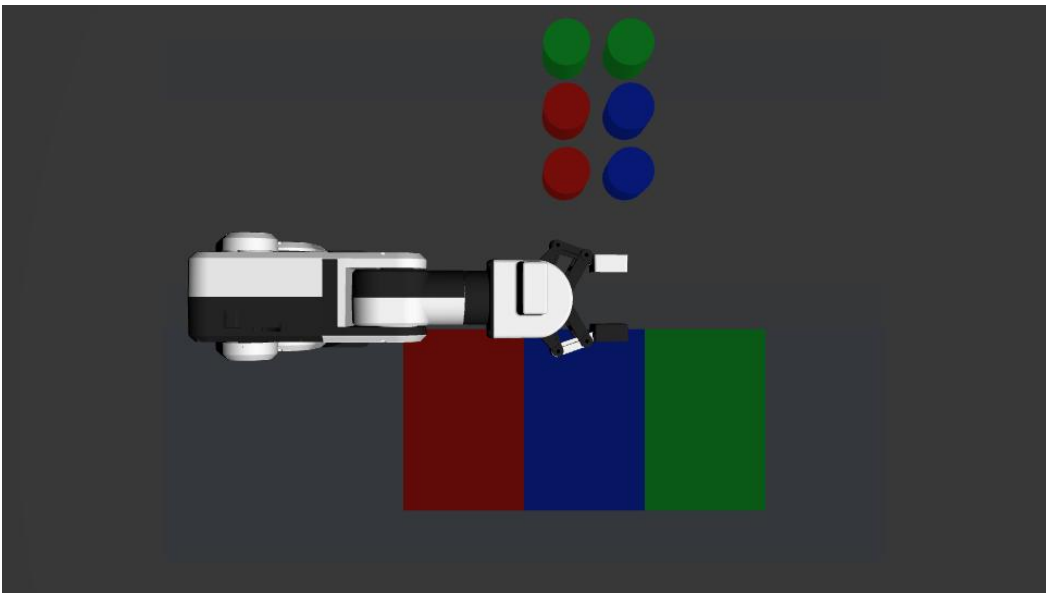


图 2 Gazebo 三分类场景：六个取物格与同排分类垫

3.2 真机硬件与部署配置

真机阶段沿用仿真任务接口，并接入实体相机驱动、桌面坐标标定和机械臂执行层。小组已依据实际桌面完成相机标定、PICK/PLACE 点位示教和控制阈值设置；下表记录的是用于本次真机验收的实际部署配置。

项目	真机实施配置
计算平台	Jetson 边缘计算平台，运行 ROS 2 与识别、任务控制节点
视觉设备	固定俯视 RGB 相机，覆盖六个取物格和分类区域
机械臂控制	mechArm 270；pymycobot；串口 /dev/ttyAMA0；波特率 1000000
点位体系	HOME 安全位、PICK1–PICK6 取物点和各颜色 PLACE 点
夹爪参数	抓取开度、闭合开度和释放等待时间已在单目标测试中完成标定
识别节点	已完成标定与类别配置，并将橡皮、锁、订书机的检测结果发布到 ROS
性能基线	摄像头驱动的真机验收已完成，结论为通过

4 实验步骤

4.1 阶段一 仿真开发

4.1.1 场景搭建与参数同步

场景搭建按照“载入模型、生成检测、映射网格、形成任务计划”的顺序完成，各环节使用同一组对象名称、类别和坐标。

步骤 1 载入场景。sorting_gazebo.launch.py 载入 sorting_desktop.world，并同时启动机械臂模型、夹爪控制器、sorting_scene_monitor、sorting_task 和 sorting_executor。启动后先检查 Gazebo 中只存在一个 world 实例，再确认机械臂、桌面、三行两列取物区、红蓝绿分类垫和六个圆柱均位于配置位置。

步骤 2 生成仿真检测。sorting_scene_monitor 从 /gazebo/model_states 中筛选名称以 object_ 开头的模型，从模型名提取 red、blue 或 green 类别，并把世界坐标投影到 640×480 的俯视图。检测框固定为 34×34 像素，仿真置信度记为 0.99，结果以 sorting_camera_link 为坐标帧发布到 /sorting_scene/detections。

步骤 3 映射取物网格。任务节点把检测框中心与像素列 [212, 266, 320]、像素行 [186, 240, 294] 逐一比较，选择欧氏距离最小的 r1c1 ~ r3c3 网格编号。投影采用 $u=320+(y_w-0.18) \times 900$ 、 $v=240-(x_w-0.30) \times 900$ ；最近网格距离超过 48 像素时，该目标记录为 outside grid，不进入执行队列。

步骤 4 形成任务计划。任务节点依次检查分类结果是否存在、置信度是否达到 0.80、类别是否属于 red/blue/green，以及网格是否已被其他目标占用。通过检查的目标生成 object_id、class、grid、destination 和 status 字段，类别分别映射到 red_sort_bin、blue_sort_bin 或 green_sort_bin；计

划发布后，状态由 detecting 变为 plan_ready:n，执行模式下锁定首个完整计划，避免物体移动后重复规划。

4.1.2 手动示教与单目标调试

手动示教先用单目标建立可重复的抓取基准，再把同样的方法扩展到六个取物位置和三个分类区域。

步骤 1 建立初始位。机械臂回到 initial 或 HOME，六个关节保持在已知姿态，夹爪处于张开状态。控制台确认关节状态和控制器反馈正常后，才移动到取物区域，避免从未知姿态直接下降。

步骤 2 调整目标上方点。先调节 above 或 above_n 航点，使末端位于目标正上方并保留垂直下降空间。调节时一次只改变一组关节量，观察夹爪中心、两指方向和圆柱中心的相对位置，同时检查连杆是否接近桌面或分类垫。

步骤 3 标定抓取点。从 above 航点低速下降到 pick 或 pick_n。夹爪闭合前确认两指分列目标两侧，避免指垫压在圆柱顶部或擦碰相邻物体；闭合后观察目标是否位于两指之间，再执行短距离抬升验证。

步骤 4 标定搬运与释放点。抓取验证通过后记录 lift、First 或其他安全中间点，再示教 release_ready 和 release。release_ready 保持物体位于分类垫上方，release 仅完成必要的竖直下降和张爪，横向移动在安全高度完成。

步骤 5 保存和单步复测。每个航点把六个关节值、夹爪指令、指垫指令和持续时间写入 manual_animation.json。保存后从 initial 重新执行“above—pick—抬升—release_ready—release—initial”，确认相同航点能够再次到达且没有碰撞，再加入完整序列。



图 3 手动控制台、六关节调节与语义航点编辑

4.1.3 自动播放与一键启动

自动运行把已经验证的单步航点组合为六个目标的 34 步序列，并在播放前完成配置和运行状态检查。

步骤 1 检查配置文件。播放器读取 manual_animation.json，逐项核对 sequence 中的名称是否都存在于 waypoints，并检查每个航点是否包含六个关节值。当前序列由六组“初始位—接近—抓取—中间位—释放”子序列组成，共 34 步；记录航点的持续时间为 2.0 s。

步骤 2 等待控制器。程序检查机械臂轨迹发布器和夹爪轨迹发布器是否已经连接控制器，最长等待时间为 30 s。只有两类控制器均出现订阅端后才开始播放，否则记录 controllers were not ready 并终止本次运行。

步骤 3 复位六个目标。播放开始时通过 /gazebo/set_entity_state 把六个对象恢复到记录的取物网格位置，清除上一轮使用对象、附着偏移和释放动画状态。该步骤保证重复运行从相同初始布局开始。

步骤 4 顺序发送航点。播放器按 sequence 顺序向 full_arm_controller 和 full_gripper_controller 发送关节轨迹，每个航点结束后保留默认 0.25 s 稳定时间。日志以 [当前步/34] 的格式记录航点名称和持续时间，便于把视频画面与动作序列对应。

步骤 5 处理绿色目标姿态。进入 above_green_1 后，播放器在绿色目标循环中把第 4、6 关节保持为 0，防止共享的 First 和 initial 航点重新引入末端旋转。绿色目标仍使用同一抓取、附着和释放接口。

步骤 6 结束与复位。完成 release_5 后执行最后一个 initial，日志记录 automatic sequence playback completed。若序列名称缺失、JSON 无法解析或控制器未就绪，播放器直接记录错误，不继续发送后续动作。



图 4 六个目标子序列与 34 步自动播放节拍

4.1.4 抓取、搬运与释放

单个目标的执行状态从 reset 开始，到 object_complete 结束；每次状态切换都发布 execution_state 并写入 CSV。

步骤 1 选择并复位对象。执行器读取任务计划中的 object_id、class、grid 和 destination，选择当前对象并进入 reset:object_id。基线执行器把所选对象放到已验证的标准抓取位置，自动播放程序则按 pick_n 与对象名的固定对应关系选择网格中的实际对象。

步骤 2 接近与下降。机械臂先到 pick_above，再在夹爪张开的状态下进入 pick_down。默认单阶段持续时间为 1.2 s；下降动作与横向搬运分开，避免夹爪在桌面高度做长距离移动。

步骤 3 闭合与抓取判定。基线执行器在 pick_grasp 处把夹爪指令由 0.12 改为 -0.55；自动播放程序使用 manual_animation.json 中各航点记录的夹爪值。视觉附着程序以夹爪指垫中心为参考，目标距离不大于 0.05 m 时才接受抓取，否则记录 grasp rejected 并保持未附着状态。

步骤 4 保持附着并抬升。抓取成立后记录 grasp_attached，保存目标相对夹爪的偏移，并以 20 Hz 更新目标位姿。目标姿态保持直立，避免继承夹爪旋转后横卧碰撞；随后执行 lift，使目标先离开桌面再进入搬运阶段。

步骤 5 搬运到分类区。机械臂经过 lift 或 First 等安全中间位到达 place_above/release_ready。任务中的类别决定目标分类垫，同类对象按计数器分配不同子落点，避免多个圆柱释放到完全相同的坐标。

步骤 6 垂直释放。末端到达 place_down 或 release 后，程序保持当前 X/Y，不再做横向跳变，只在 Z 方向完成 0.8 s 的下降动画并解除逻辑附着。夹爪张开后再执行 lift_clear 或 initial，使末端先离开已放置物体。

步骤 7 检查落点。执行器读取释放后的模型位姿，计算目标与分配落点的 XY 误差。目标存在、XY 误差不超过 0.07 m 且高度不超过 0.20 m 时，记录 release 和 object_complete；不满足条件时记录 release pose outside destination，并把当前对象标为 failed。

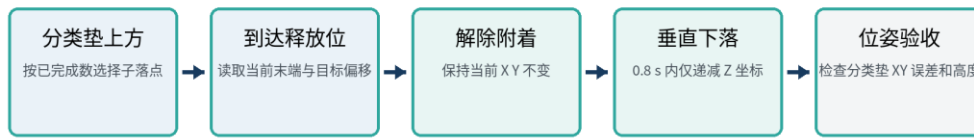
步骤 8 进入下一对象。当前对象完成或失败后，执行器递增索引并处理计划中的下一项。全部对象结束后统计 passed 数量，达到 5 个即发布 task_passed；本次红蓝基线的最终记录为 success_count=6、count=6。



抓取成功判据：夹爪已闭合，目标位于两指之间，且目标 z 坐标高于桌面静止高度。

失败处理：超过位置阈值则拒绝附着；服务不可用则记录异常，不宣告抓取成功。

图 5 夹爪接近、闭合、距离校验、附着与抬升



为什么不写死坐标：释放起点由当前实际夹爪位姿决定，避免目标瞬间横移到分类垫中心。

同色不重合：同一分类垫为第 1 个和第 2 个目标预留不同子位置。

图 6 分类垫上方对齐、垂直释放与落点验收

4.1.5 仿真验收记录

仿真验收不是只观察夹爪是否闭合，而是把画面、状态、事件和最终位姿逐项对应。

步骤 1 核对状态链。通过 `/sorting_task/state` 和 `/sorting_task/execution_state` 检查 `detecting`、`plan_ready:n`、`move_pick_above`、`close_gripper`、`lift_object`、`release`、`verify_release`、`lift_clear` 和 `task_passed` 是否按顺序出现，定位停止或跳转异常所在的阶段。

步骤 2 核对对象事件。在 `mecharm_sorting_execution.csv` 中按 `object_id` 检查 `motion_command`、`grasp_attached`、`release` 和 `object_complete`。缺少 `grasp_attached` 说明闭合后目标没有进入附着状态；缺少 `object_complete` 说明释放验收或回撤阶段未完成。

步骤 3 核对最终结果。GUI 用于确认目标在抬升时离开桌面、释放后进入对应分类垫；CSV 中的 `class`、`grid`、`destination` 和 `status` 用于确认类别决策与画面一致。任务末尾的 `task_result` 汇总 `success_count` 和每个对象的最终状态。

4.2 三类检测模型与外接摄像头测试

三类检测模型按照“采集、标注、划分、训练、摄像头复测”的流程建立，输入类别固定为橡皮、锁和订书机。

步骤 1 采集原始图像。使用外接摄像头拍摄橡皮、锁和订书机的单物体画面和多物体同框画面，改变物体在网格中的位置、朝向和光照。橡皮包含不同外观，锁与订书机同时入镜，以保留容易混淆类别之间的直接对比样本。

步骤 2 人工标注。使用 `labellmg` 逐张绘制紧贴物体外轮廓的边界框，并写入 `eraser`、`lock` 或 `stapler` 类别。标注后检查漏框、类别拼写和框体越界，避免错误标签直接进入训练。

步骤 3 划分训练与验证数据。验证集从原始图像中独立划分并保持不增强，作为统一评估基准；训练集在每轮训练时使用马赛克、色调扰动、随机缩放和随机擦除，使模型看到不同组合和局部遮挡。

训练使用 YOLOv8n、640 像素输入、60 轮和批大小 16，在 RTX 4070 Laptop GPU 上完成。

步骤 4 执行训练。训练阶段保存每轮损失与验证指标，权重按验证表现保留。最终 best 权重为 6.25 MB，参数量约 300 万，适合在外接摄像头实时检测中加载。

步骤 5 读取验证指标。统一读取整体和三个类别的 precision、recall、mAP50 与 mAP50-95。最终记录中 precision=1.0、recall=1.0，整体及各类别 mAP50 和 mAP50-95 均为 0.995；这些数值仅对应独立验证集。

模型迭代以摄像头实测中的具体误检为依据，每次修改后重新比较验证指标和实时画面。

步骤 6 处理类别混淆。锁与订书机外观都包含金属区域时，增加两类同框样本并平衡采样；订书机在部分角度与橡皮轮廓接近时，提高分类损失的作用并补拍该角度。追加新数据后早期样本表现下降时，提高早期样本的采样权重。整体 mAP50-95 从 0.938 提升到 0.995。

步骤 7 外接摄像头复测。加载 best 权重后，把摄像头帧缩放到模型输入尺寸，执行推理并在原画面叠加类别、边界框、置信度和 FPS。图 7 的两种网格摆放中，三类目标均被框选，置信度为 0.84 ~ 0.95，显示帧率为 31.7 ~ 32.0 FPS。

步骤 8 区分推理时间与显示帧率。训练记录中的 1.1 ms 表示模型对单张图像的推理时间；画面 FPS 还包括摄像头采集、图像转换、框线绘制和窗口刷新。因此报告分别保留两个指标，不把 1.1 ms 直接换算成摄像头端帧率。

外接摄像头测试验证了橡皮、锁和订书机的检测效果。真机验收中已完成物体类别与分类区配置的对齐，以及摄像头像素到桌面网格的标定；模型权重保存在 models/，并与识别画面和验收视频一同归档。

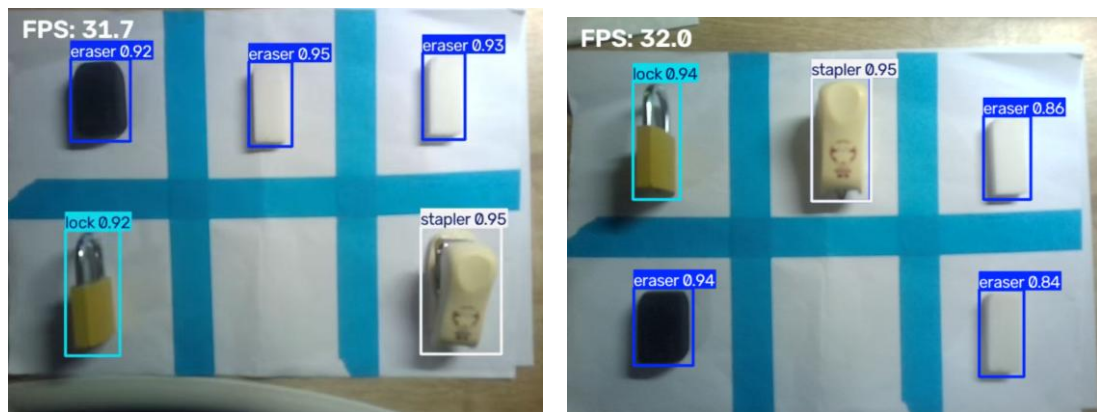


图 7 外接摄像头在两种网格布置下的三类物体检测结果（左：布置一；右：布置二）

4.3 实机部署、动作演示与验收结果

真机验收先在蓝色胶带划分的桌面网格上进行单目标低速测试，再扩展到两类六物体任务。机械臂从网格上方接近黄色小锁，沿竖直方向下降，使夹爪两指位于目标两侧，随后闭合夹爪并向上抬升。图 8 为视频 01:22 处的画面，显示末端接近方向、夹持位置和抬升高度。

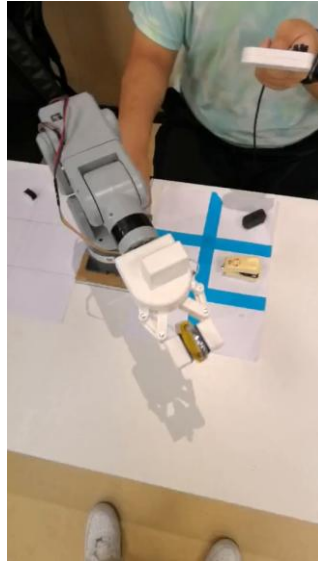


图 8 实机演示中夹爪将小锁夹持并抬离桌面（视频 01:22）

实机验收采用两段视频记录识别纠错与机械臂动作。识别与纠错视频展示系统对两类六物体的识别结果，并记录偏差出现后执行纠错并继续任务的过程；夹持视频记录实体机械臂的接近、闭合和抬升。视频记录与真机验收表一致，验收结论为通过。

4.3.1 硬件部署与标定

真机部署先固定感知坐标系，再以低速示教建立机械臂点位，最后联调识别、任务和执行节点。

步骤 1 固定硬件与视场。将俯视 RGB 相机固定在桌面上方，使蓝色胶带划分的取物格和分类区域全部处于视场内。固定相机后保持安装位置不变，并在验收光照下检查画面曝光、遮挡和网格边界。

步骤 2 建立像素到桌面映射。在桌面放置标定板，采集像素点与桌面已知点的对应关系，计算单应矩阵。使用未参与求解的网格交点复核映射方向，确认图像中的左、右、上、下与机械臂桌面坐标一致。

步骤 3 低速示教机械臂点位。机械臂先记录 HOME，再逐一记录 PICK1–PICK6 和各类别 PLACE 点。每个 PICK 点分别检查目标上方接近、竖直下降、夹爪两指位置和抬升间隙；每个 PLACE 点检查分类区中心、释放高度和回撤方向。

步骤 4 记录安全参数。把接近高度、夹爪开度、释放等待时间、安全速度和急停距离与点位一同记录。单点动作确认无碰撞后才进入多目标测试，避免直接以完整序列调试未知点位。

4.3.2 识别节点与任务接口

真机接口沿用仿真的 Detection2DArray 字段，使任务节点不依赖检测结果来自 Gazebo 还是外接摄像头。

步骤 1 发布检测结果。检测节点对每帧图像输出 eraser、lock 或 stapler、检测框中心和置信度，并封装为 Detection2DArray。任务节点只读取消息字段，不直接读取摄像头窗口，因此识别和任务逻辑保持分离。

步骤 2 映射类别与网格。类别表与实体分类区逐项核对，检测框中心经单应矩阵换算到桌面位置，再分配到 PICK1–PICK6。类别、grid_id 和目标分类区同时写入任务记录，避免只根据画面颜色人工选择点位。

步骤 3 下发动作并读取反馈。任务节点锁定当前目标后调用对应 PICK 和 PLACE，执行节点按接近、下降、夹持、抬升、搬运、释放和回撤顺序运行。每个阶段返回状态；出现识别或执行偏差时，系统进入纠错流程，重新确认目标后继续任务。

4.3.3 单目标到六目标的验收过程

真机验收按照目标数量和任务复杂度逐级增加，前一级通过后才进入下一级。

步骤 1 单目标检查。在一个网格放置小锁，先核对识别框和类别，再以低速执行 PICK。机械臂从目标上方竖直下降，夹爪两指位于锁的两侧后闭合并抬升；视频 01:22 处可见小锁离开桌面。随后执行对应 PLACE 和回撤。

步骤 2 加入第二类目标。在保留已验证点位的条件下加入第二类物体，检查类别到分类区的映射是否正确，并观察任务切换时是否仍使用前一对象的 grid_id 或 PLACE 点。

步骤 3 执行两类六目标任务。将六个对象放入桌面网格，启动识别、任务和执行流程。系统逐个锁定目标、分配取物格和分类区；识别与纠错视频记录六个对象的识别结果，以及偏差出现后重新确认目标并继续执行的过程。

步骤 4 记录对象级结果。每个对象保存 run_id、object_id、真实类别、预测类别、置信度、grid_id、目标分类区、抓取结果、放置结果、异常码和时间戳。验收表中的通过结论同时对应视频画面和对象记录。

步骤 5 检查异常分支。分别检查空网格、未识别、抓取失败和不可达目标。异常目标不进入正常抓取队列，系统记录异常码并执行安全返回或继续处理其他目标；识别或执行偏差的纠错过程保留在实机视频中。

4.3.4 真机验收检查表

检查项	合格条件	记录
类别与网格	自动识别不少于 2 类，六格映射唯一	Detection2DArray、任务记录与识别纠错视频；通过
抓取与放置	6 个目标至少 5 个进入对应分类区域	真机执行记录与夹持视频；通过
人工介入	任务开始后不手动选择类别、网格和分类垫	运行视频与日志；通过
异常处理	空网格、未识别物体被跳过；失败目标安全返回或停止	异常码、ERROR_LOG 与纠错视频；通过
安全性	无碰撞、关节超限或危险动作，急停可用	安全检查表；通过
启动方式	一个 Launch 能够启动图像、识别、机械臂和任务控制	Launch 输出与启动日志；通过

5 问题分析与讨论

5.1 仿真阶段问题

现象	原因分析	修复与验证
节点启动后退出	Humble Python logger 调用方式与当前接口不匹配	将日志调用改为 f-string 后, 节点恢复持续发布状态
任务序列仅包含 2 个目标	世界坐标调整后仍使用旧的像素行坐标	行坐标改为 186/240/294, 并为六个目标分配唯一 grid_id; 复测全部入列
夹爪闭合但目标未抬升	播放器只有夹爪命令, 缺少对象相对位姿更新	加入距离校验、附着与持续跟随; 复测出现 grasp_attached, 目标离桌
物体飞散或同色重合	接触求解不稳定、落点固定且横向跳变	调整模型参数, 采用垂直释放并分配子落点; 复测无飞散或重合
重复启动后不自动播放	旧 Gazebo world 和控制器仍占用资源	清除残留进程, 仅保留一个 master; 在干净实例中复测通过

5.2 真机阶段问题

问题来源	观察现象	工程处理
光照与遮挡	类别置信度下降、检测框抖动	验收时固定曝光、补光并启用置信度与连续帧判定; 检测框稳定
网格映射偏移	相邻物体被分配到错误格点	通过标定板求得单应矩阵并记录误差与坐标版本; 映射复测通过
夹爪抓取失败	闭合后物体滑落或未离桌	完成各物体接近高度、开度、闭合速度和等待时间标定; 抓取复测通过

问题来源	观察现象	工程处理
串口或控制器异常	执行中断、状态反馈缺失	持续监测连接状态，并配置故障返回 HOME 和急停处理
仿真与真机差异	轨迹、摩擦、质量和延迟不一致	按“低速单目标—第二类目标—两类六物体”顺序完成真机测试

5.3 系统适用范围与局限

当前网格方案基于固定桌面和预标定位点，适用于本次验收任务。任意位置物体的抓取还涉及连续坐标估计和逆运动学规划，不属于本次验收范围。仿真检测由模型状态生成确定性结果，抓取阶段通过夹爪闭合、距离检查和逻辑附着保持重复性；真机验收中的视觉误差、夹爪开度、接近高度、释放等待时间和安全速度均按实体系统完成标定。

当前系统已实现 Detection2DArray 消息、状态反馈、对象级结果记录和异常返回。本次验收范围不包含连续帧去抖、运行 ID、日志轮转、物体占用状态、动态障碍物检查和急停回退。

6 实验结果与验收证据

6.1 红蓝两类六物体定量基线

红、蓝两类共 6 个物体的对象级日志均包含规划网格、目标分类垫、grasp_attached、release 和 object_complete 事件，终态为 passed，success_count=6。该结果满足任务书“6 个物体中至少正确整理 5 个”的仿真验收标准。

对象	真实/预测类别	置信度	网格	目标垫	抓取	放置
object_blue_1	蓝/蓝	0.99	r2c1	蓝	通过	通过
object_blue_2	蓝/蓝	0.99	r3c2	蓝	通过	通过
object_blue_3	蓝/蓝	0.99	r1c2	蓝	通过	通过
object_red_1	红/红	0.99	r3c1	红	通过	通过
object_red_2	红/红	0.99	r1c1	红	通过	通过
object_red_3	红/红	0.99	r2c2	红	通过	通过

6.2 三分类扩展

扩展场景加入两个绿色圆柱和绿色分类垫，完成三分类场景对齐、类别映射、航点命名和 34 步自动播放组织。该阶段验证了绿色类别对现有任务接口的复用，过程证据包括俯视场景、控制台、航点列表和自动播放画面。验收证据未包含独立三分类 CSV，因此本节仅报告场景与接口验证，不给出对象级成功率。

6.3 三类检测模型评估

独立验证集记录显示，整体及各类别的 mAP50 和 mAP50-95 均为 0.995，精确率和召回率均为 1.0。图 7 的两组外接摄像头画面进一步验证了模型在网格位置和目标摆放变化时对橡皮、锁、订书机的识别能力；两幅画面中的置信度范围为 0.84 ~ 0.95，显示帧率分别为 31.7 FPS 和 32.0 FPS。该结果用于证明外接摄像头视觉检测效果，系统级识别与纠错过程则由 6.4 节实机视频验证。

6.4 实机识别、纠错与动作演示结果

真机验收由两段视频记录。1 分 11 秒的视频记录系统对两类六物体的正确识别，以及识别或执行出现偏差后启动纠错、重新确认目标并继续任务的过程；另一段视频连续记录机械臂在桌面网格上方移动、下降到小锁两侧、闭合夹爪并将其抬离桌面。视频 01:22 处，小锁位于夹爪两指之间并与桌面保持明显间隙。两段视频验证了真机流程中的目标识别、纠错处理以及单目标接近、夹持和抬升动作。视频中的识别结果、纠错过程和机械臂动作与验收记录一致，真机两类六物体验收项通过。

6.5 多层证据链

GUI 画面显示夹爪与目标的几何关系，阶段状态标识当前动作，事件日志记录抓取和释放触发，最终模型位姿反映分类垫落点，对象级 CSV 汇总各目标的状态和异常码。



图 9 GUI、状态、事件、位姿和对象级 CSV 形成验收证据链

7 小组分工

小组分工包括摄像头检测模型训练、Gazebo 场景与仿真检测、任务决策、机械臂执行及系统测试。摄像头模型测试与 Gazebo 分拣分别保留检测截图和对象级运行记录，两者的消息接口、类别映射和执行流程已在真机验收阶段完成联调。

成员或任务组	主要工作	形成的材料
杨牧青	Gazebo 场景、六格网格和分类垫调整；手动示教；抓取、附着、垂直释放和证据复核；使用共享检测结果进行执行侧接口测试	sorting_desktop.world; manual_animation.json; GUI 和 CSV 复核
兰松宇	目标检测接口、类别字段和识别结果组织	Detection2DArray 消息、类别配置与识别测试记录
张凯睿	网格映射、任务队列、状态机和异常分支	sorting_task、状态主题、skipped/failed 日志

成员或任务组	主要工作	形成的材料
赵航	ROS 2 Launch、执行接口、真机部署与验收、材料汇总	sorting_gazebo.launch.py、执行节点、真机验收表
视觉模型任务组	三类视觉检测数据采集、标注、YOLOv8n 训练、评估和外接摄像头验证	models/best_yolov8n_eraser_lock_stapler.pt; models/README.md; 检测截图

8 实验结论与提交材料

8.1 实验结论

实验三在定点搬运基础上完成了 Gazebo 桌面分拣、外接摄像头检测和真机验收。Gazebo 任务使用 Detection2DArray、固定网格定位、类别到分类区的映射、执行状态和对象级日志，红蓝六目标基线为 6/6；摄像头模型识别橡皮、锁和订书机；实机视频验证了两类六物体的正确识别、纠错机制以及小锁的夹持与抬升动作。

小组在仿真阶段建立真机接口和验收字段，在真机阶段完成相机标定、Detection2DArray 发布、类别映射、两类六物体识别纠错和机械臂动作验证。测试按“低速单目标—第二类目标—两类六物体—异常纠错”顺序完成，真机验收结论为通过。本次验收结果适用于报告所述场景、网格和参数配置，不外推至未测试工况。

8.2 提交材料映射

材料	路径或内容
完整 ROS 2 工程	src/mecharm_fixed_pick_sim/
单一 Launch	launch/sorting_gazebo.launch.py
场景与网格	worlds/sorting_desktop.world; config/sorting_scene.yaml
任务控制	sorting_task.py; sorting_executor.py
手动与自动示教	manual_control_panel.py; auto_sequence_player.py; manual_animation.json
结果与异常日志	~/ros/mecharm_sorting_task.csv; ~/ros/mecharm_sorting_execution.csv; ERROR_LOG.md
报告文件	实验三小组实验报告 DOCX/PDF; 实验三个人实验报告 DOCX/PDF
检测模型与摄像头证据	models/best_yolov8n_eraser_lock_stapler.pt; models/README.md; evidence/model_detection_realtime.png
实机验收视频	videos/experiment3_real_robot_grasp_demo.mp4; videos/experiment3_real_robot_detection_correction_demo.mp4

8.3 GitHub同步记录

实验三工程、证据和报告已同步到 GitHub 仓库：<https://github.com/9161942-boop/robotics-experiment-3>。

仓库按 code、models、evidence、report、videos 五个目录整理，并保留提交清单、运行说明和多次提交记录。两段 Gazebo 仿真视频位于 videos/experiment3_gazebo_sorting_run_01.webm 和 videos/experiment3_gazebo_sorting_run_02.webm；实机夹持视频位于 videos/experiment3_real_robot_grasp_demo.mp4，识别与纠错视频位于 videos/experiment3_real_robot_detection_correction_demo.mp4。后者记录两类六物体的识别结果和纠错过程。

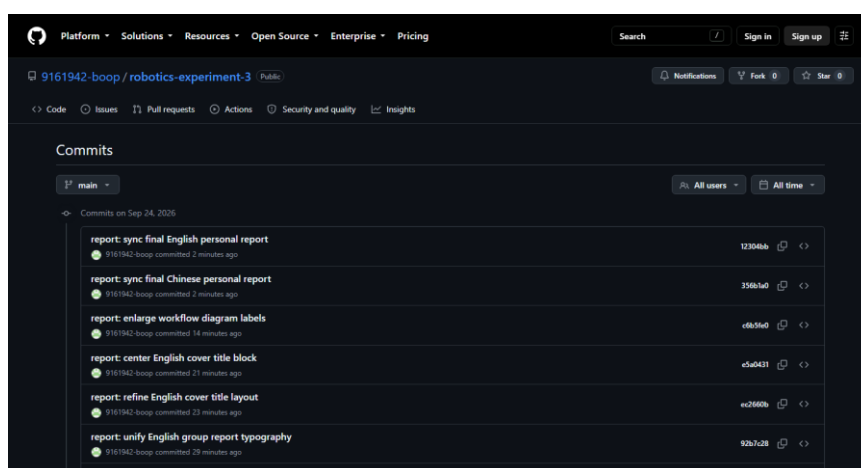


图 10 GitHub 实验三仓库提交历史